

32. Hodnocení složitosti algoritmů (paměťová a časová složitost, asymptotická složitost)

32. Hodnocení složitosti algoritmů

SZZ okruh 32 (FIT BUT). Měření náročnosti algoritmů nezávisle na stroji.

Shrnutí

Časová a prostorová složitost

- **Časová složitost** = řádový počet **elementárních operací** v závislosti na velikosti vstupu n (ne reálný čas).
- **Prostorová složitost** = spotřeba paměti; obvykle menší než časová (často $O(1)$; Quick $O(\log n)$ zásobník, Merge rekurze, BFS $O(b^d)$).
- Postup: určit $n \rightarrow$ max. počet operací $\rightarrow$ ponechat nejrychleji rostoucí člen $\rightarrow$ škrtnout konstanty.

Asymptotická notace

- **O (Omikron)** — horní hranice (nejčastější); **Ω (Omega)** — dolní hranice; **Θ (Théta)** — přesná třída (obě hranice).
- Typické třídy: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$.
- **P** (řešitelné v polynomiálním čase) $\times$ **NP** (ověřitelné/nedeterministicky polynomiální).

Určování složitosti

- **Vnořené cykly** $\rightarrow$ násobení (n^2 , n^3); dělení řídicí proměnné $\rightarrow \log n$.
- **Rekurze** $\rightarrow$ často exponenciální (faktor větvení: 2 volání $\rightarrow 2^n$); velký vliv i na **paměť** (ukládají se lokální proměnné každého volání).

Použití u řazení/vyhledávání viz [[okruhy/30-vyhledavani-razeni]]; u prohledávání viz [[okruhy/26-reseni-uloh-prohledavani]].

Související syntéza

- [[synthesis/slozitost-napric-algoritmy | Asymptotická složitost napříč algoritmy]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Základní pojmy $\rightarrow$ [[#Časová a prostorová složitost]] - *Časová složitost* — co měří? $\rightarrow$ Počet elementárních operací v závislosti na velikosti vstupu (ne čas v sekundách). - *Prostorová složitost*? $\rightarrow$ Spotřeba paměti v závislosti na vstupu; rekurze ji výrazně zvyšuje.

Asymptotická notace - O vs. Ω vs. Θ ? - O horní hranice, Ω dolní hranice, Θ obě (přesná třída). - *Typické třídy + příklady?* - $O(1)$ index pole, $O(\log n)$ binární vyhledávání, $O(n)$ průchod, $O(n^2)$ bubble sort, $O(2^n)$ rekurze.

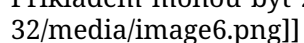
Určování složitosti - *Vnořené cykly?* - Násobí se - $O(n^2)$, $O(n^3)$; dělení řídicí proměnné - $\log n$. - *Proč se konstanty vynechávají?* - Asymptoticky ($n \rightarrow \infty$) nehrají roli; podstatný je řád růstu.

Plné znění (ke studiu)

Složitost algoritmů definuje kritéria, podle kterých jednotlivé algoritmy hodnotíme. Nejčastějšími kritérii jsou **časové a paměťové nároky algoritmu**. Reálné doby běhů programů se **liší na různých strojích** a s různými vstupy. Proto zavádíme **časovou a prostorovou složitost algoritmů**, které umožňují:

- vyjádření vlastností algoritmu **nezávisle** na technických podrobnostech (počítači, na kterém běží),
- popis chování algoritmu pomocí **jednoduchých matematických funkcí**.

Časová složitost

Časová složitost je odvozena od počtu elementárních operací (sčítání, násobení, porovnání, ...). Udává **řádový počet provedených elementárních operací v závislosti na velikosti vstupu** (počet prvků řazeného pole, větší z čísel, u kterého hledáme společného dělitele, ...). Příkladem mohou být 2 různé (nejedná se o stejnou funkcionalitu) algoritmy součtů: 

- **Součet1:** U prvního algoritmu lze jednoduše **detekovat cyklus** podle **n** a **vnořený cyklus** podle **n** (obecně nemusí být jednoduché vnořené cykly detekovat). To znamená, že pro **každé konkrétní číslo z n** se provede **n iterací**, celkově tedy **n*n iterací**, neboli **n² iterací**, a v každé iteraci je **provedeno sčítání**.
- **Součet2:** Zde je pouze **jeden cyklus** (while), tudíž hned na první pohled by se dalo **chybně** říct, že bude provedeno **n operací**. To ale není pravda, protože je nutné si v těle cyklu všimnout **dělení n** v každé iteraci, to znamená, že bude provedeno **log₂(n)** operací.

Postup určení časové složitosti

1. Zhodnotíme, co je vstupem **n** a jak **měřit jeho velikost**.
2. Určíme maximální možný počet elementárních kroků algoritmu (elementárních operací jako je sčítání) provedených pro vstup o **velikosti n**.
3. Ve výsledné formuli **ponecháme pouze nejrychleji rostoucí člen**, ostatní zanedbáme.
4. Seškrtneme multiplikativní konstanty (**odebereme násobení konstantou**).

Průměrná časová složitost

Určujeme, pokud pro **různé vstupy stejné velikosti** vykoná algoritmus **různý počet kroků** (např. řazení seřazeného pole, náhodně uspořádaného pole a opačně seřazeného pole). Za časovou složitost poté považujeme **aritmetický průměr** časových nároků **přes všechny vstupy** (docela nereálné) dané velikosti. Proto u příkladu se řazením používáme jinou vlastnost, a to přirozenost algoritmu.

Asymptotická časová složitost

Jedná se o nejčastěji používané hodnotící kritérium, vychází z toho, že pro **malá n** se **nejrychleji rostoucí člen** nemusí výrazně projevovat. Popisujeme tedy chování algoritmu pro **n blíží se k nekonečnu**. Používají se tři různé asymptotické složitosti:

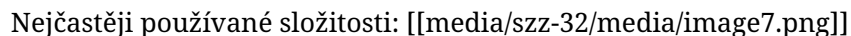
- O – Omikron (velké O , $\mathcal{O}$, big O) – horní hranice chování (**nejčastější**),
- Ω – Omega – dolní hranice chování,
- Θ – Théta – třída chování.

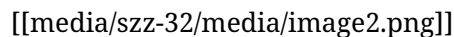
U definování složitostí se snažíme, aby co nejvíce odpovídaly složitosti algoritmu (tj. hledáme **nejmenší horní** hranici a **největší dolní** hranici). Není podstatná konkrétní přesná časová závislost, stačí **charakterizovat třídu**.

Složitost Omikron - O

Vyjadřuje **horní hranici** časového chování algoritmu (omezuje chování algoritmu shora, tj. algoritmus **nebude nikdy pomalejší (nebude trvat déle)**, než dané omezení). Značíme jako **Omikron($g(n)$)** nebo **$O(g(n))$** nebo jen **$O(g(n))$** .

- Zápis **$f(n) \in O(g(n))$** , označuje, že funkce **$f(n)$** je **rostoucí maximálně tak rychle** jako funkce **$g(n)$** . Dostatečně velký **násobek funkce $g(n)$** shora omezuje funkci **$f(n)$** pro **dostatečně velké n** .

Nejčastěji používané složitosti: 



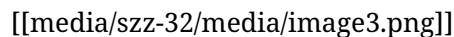


Složitost Omega - Ω

Složitost Omega vyjadřuje **dolní hranici** časového chování algoritmu (omezuje chování algoritmu zdola, tj. algoritmus **nebude nikdy rychlejší (nebude trvat kratší dobu)**, než dané omezení). V praxi nemá moc využití. Značíme jako **Omega($g(n)$)**, **$\Omega(g(n))$** nebo jen **$\Omega(g(n))$** .

- Zápis **$f(n) \in \Omega(g(n))$** , označuje, že funkce **$f(n)$** je **rostoucí minimálně tak rychle** jako funkce **$g(n)$** . Funkce **$g(n)$** je tak **dolní hranicí množiny všech funkcí**, určených zápisem **Omega($g(n)$)**.

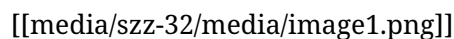
Složitost Théta - Θ



Složitost **Theta** vyjadřuje třídu chování algoritmu – ohraničuje funkci $f(n)$ z obou stran (slouží jako **dolní i horní hranice**, tedy že funkce nebude **nikdy rychlejší** než **$c_1 \cdot g(n)$** a nebude **nikdy pomalejší** než **$c_2 \cdot g(n)$**). Značíme jako **Theta($g(n)$)**, nebo **$\Theta(g(n))$** .

- Zápis **$f(n) \in \Theta(g(n))$** označuje, že funkce **$f(n)$** roste **stejně tak rychle** jako funkce **$g(n)$** .

Třídy složitosti



- **P** - Pokud existuje Turingův stroj, který úlohu vyřeší v polynomiálním čase.
- **NP** - Pokud existuje nedeterministický Turingův stroj, který rozhodne úlohu v polynomiálním čase

Orientační rychlost výpočtu

Orientační typické hodnoty velikosti vstupu **n**, pro které algoritmus s danou časovou složitostí ještě většinou zvládne na běžném PC spočítat výsledek **ve zlomku sekundy nebo maximálně v řádu sekund**. [[media/szz-32/media/image4.png]]

Paměťová (prostorová) složitost

Udává **spotřebu paměti**, případně **diskového prostoru** v závislosti na **vstupních datech**. Prostorová složitost **nebývá kritická** a často ji neřešíme (paměť si narozdíl od času můžeme koupit). Nicméně může mít **zásadní** vliv na realizovatelnost algoritmu. Obvykle je u algoritmů ale **prostorová složitost menší než časová**. Např. u většiny řadících algoritmů je prostorová složitost **konstantní $O(1)$** . U Quick sort je **logaritmická $O(\log(n))$** - nutné si uchovat **hranice ještě nezpracovaných částí**. Stejně tak je nutné si uchovávat hranice polí u Merge sort - **rekurze znamená paměť**. U algoritmu pro prohledávání stavového prostoru **BFS** je např. prostorová složitost stejná jako časová, a to exponenciální **$O(b^d)$** (**b** je faktor větvení, **d** hloubka). U DFS je časová složitost stále exponenciální, ale prostorová pouze **$O(b \cdot d)$** . U backtracking je pak složitost pouze **lineární $O(d)$** .

Asymptotická paměťová složitost

Obdobně jako u časové složitosti se používají **tři** různé asymptotické složitosti:

- O – Omikron (velké O , $\mathcal{O}$, big O) – horní hranice chování (**nejčastější**),
- Ω – Omega – dolní hranice chování,
- Θ – Théta – třída chování.

Mají i stejný význam.

Určování složitosti

Určování časové složitosti

Hlavní dva jevy, které řešíme při určování časové složitosti, jsou **cykly** (zejména **vnořené**) a **rekurze** závisující na **vstupních datech**.

- **Vnoření cyklů** většinou znamená násobení složitosti $\square$ vede na polynomiální časovou složitost: dva vnořené cykly - **$O(n^2)$** , tři vnořené cykly - **$O(n^3)$** . Nutno **sledovat úpravy řídicí proměnné cyklu**, např. její **dělení** vede na **logaritmickou složitost $O(\log(n))$** .
- **Rekurze** většinou znamená **exponenciální složitost**. Záleží, na **počtu rekurzivních volání funkce** v jejím těle. Pro dvě volání je složitost **$O(2^n)$** , pro tři volání **$O(3^n)$** , ... Počet volání můžeme označovat jako **faktor větvení** (zejména se používá u stromů).

Při více cyklech za sebou uvažujeme ten **nejhorší** - **nejrychleji rostoucí člen**.

Určování paměťové složitosti

Musíme identifikovat, jaké používáme proměnné

- **lokální statické proměnné**,
- **dynamické proměnné**,
- **REKURZE** - má **velký vliv** na paměťovou složitost programu, při každém rekurzivním volání funkce se **ukládají všechny její lokální proměnné**.

Globální statické proměnné můžeme ignorovat (považovat za **konstantní** paměťovou náročnost), lokální statické proměnné můžeme ignorovat jen v případě, že se funkce **nevolá** rekurzivně. U **dy-namických** proměnných **musíme sledovat alokaci** v cyklech **nebo při rekurzi** a jak **velká paměť** je alokována (pokud cyklem projdeme **n krát**, a **n krát** alokujeme **n prvků**, je **prostorová složitost kvadratická** (časová je lineární)).

Další zdroje:

- <https://docplayer.cz/108481191-Prostorova-pametova-slozitost-algoritmu.html>
- Asymptotická složitost
- Třídy složitosti a Turingovy stroje

Zdroje

- SZZ okruh 32 — studijní materiály FIT BUT (szz-32.docx). Obrázky: media/szz-32/.

Navigace: [[index | • Přehled okruhů]] · □ [[okruhy/31-pravdepodobnost-statistika | 31. Pravděpodobnost a statistika]] · další: [[okruhy/33-zivotni-cyklus-sw | 33. Životní cyklus softwaru]] □